



TC2038 - Análisis y diseño de algoritmos avanzados

Grafos y Tries

Profesor - Alison Muñoz Capote
Computación - Facultad de Ingeniería

Mapa del Tema y Objetivos

Objetivos de la Sesión:

1. **Comprender** la topología de red y sus componentes.
2. **Dominar** las estructuras de representación en memoria (Matrices y Listas de Adyacencia).
3. **Implementar** algoritmos de Grafos.
4. **Entender** y **codificar** la estructura del Árbol Trie.

Índice:

1. Concepto de Grafo
2. Propiedades y Representación en Memoria
3. Implementación de Grafos en Código
4. El Árbol Trie
5. Implementación de Tries en Código

0. Observaciones del período 1

Orientaciones para el período 2

1. Respetar las orientaciones sobre la estructura y restricciones de la solución al problema.
2. Utilizar la IA como **SOPORTE** y no como sustituto de su aprendizaje. Se penalizará el uso inadecuado de la IA en la solución de sus tareas.

Columna 1

```
def kmp(text: str, pattern: str) -> list[int]:
    if not text:
        return []

    _lps = lps(pattern)

    if not _lps:
        return []

    j = 0
    _kmp = []

    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = _lps[j - 1]
        if text[i] == pattern[j]:
            j += 1

        if j == len(pattern):
            _kmp.append(i - j + 1)
            j = _lps[j - 1]

    return _kmp
```

Columna 2

```
def kmp(text: str, pattern: str) -> list[int]:
    if not text or not pattern:
        return []

    lps_array = compute_lps(pattern)
    matches = []
    j = 0

    for i, char in enumerate(text):
        while j > 0 and char != pattern[j]:
            j = lps_array[j - 1]

        if char == pattern[j]:
            j += 1

        if j == len(pattern):
            matches.append(i - j + 1)
            j = lps_array[j - 1]

    return matches
```

Columna 3

```
def shift_and_match(text: str, pattern: str):
    m = len(pattern)
    if not m or not text:
        return

    # 1. Precomputación: Máscaras de bits  $O(M + \Sigma)$ 
    # Cada caracter del alfabeto obtiene un entero donde los bits en 1
    # representan sus posiciones dentro del patrón.
    char_masks = {}
    for i, char in enumerate(pattern):
        char_masks[char] = char_masks.get(char, 0) | (1 << i)

    # El bit más significativo que indica que el patrón completo ha coincidido
    match_mask = 1 << (m - 1)

    # Estado inicial del autómata en un solo registro numérico
    state = 0

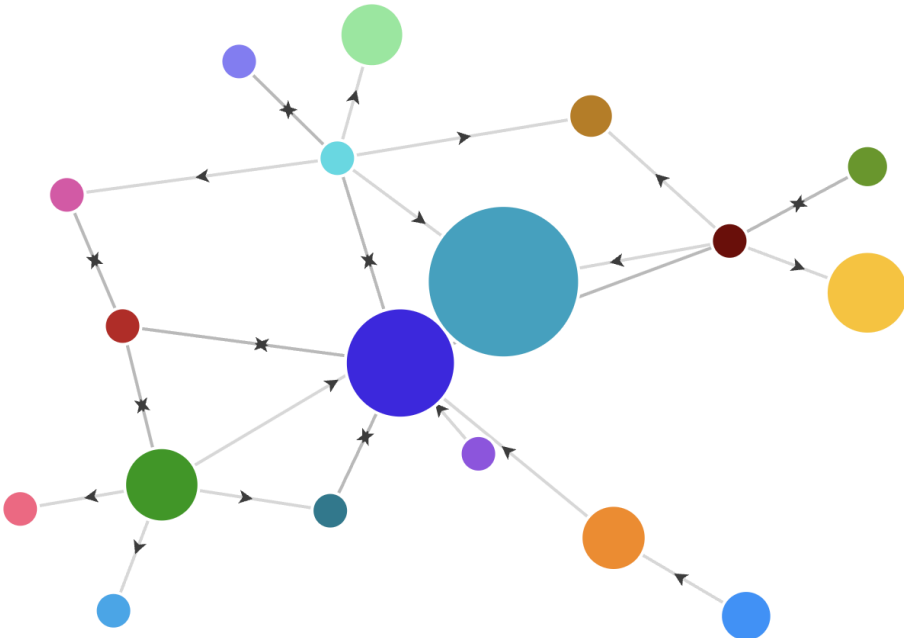
    # 2. Búsqueda:  $O(N)$  estricto. Sin bucles anidados ni backtracking.
    # state << 1: Avanza todos los prefijos posibles un paso
    # | 1: Permite que inicie una nueva coincidencia potencial
    # & char_masks: Filtra las transiciones válidas para el caracter actual
    state = ((state << 1) | 1) & char_masks.get(char, 0)

    # Si el bit en la posición m-1 es 1, llegamos al estado de aceptación
    if state & match_mask:
        yield i - m + 1
```

1. Concepto de Grafo

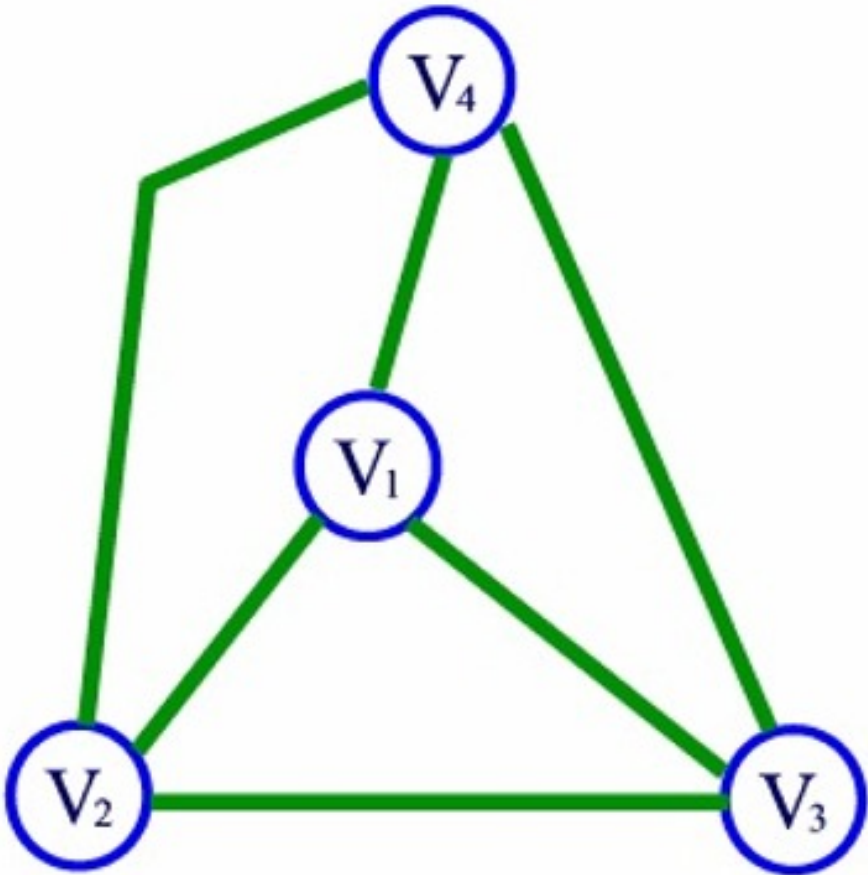
¿Qué es un grafo?

Una red de conexiones. Modela relaciones complejas donde no existe una jerarquía estricta (a diferencia de los árboles tradicionales).



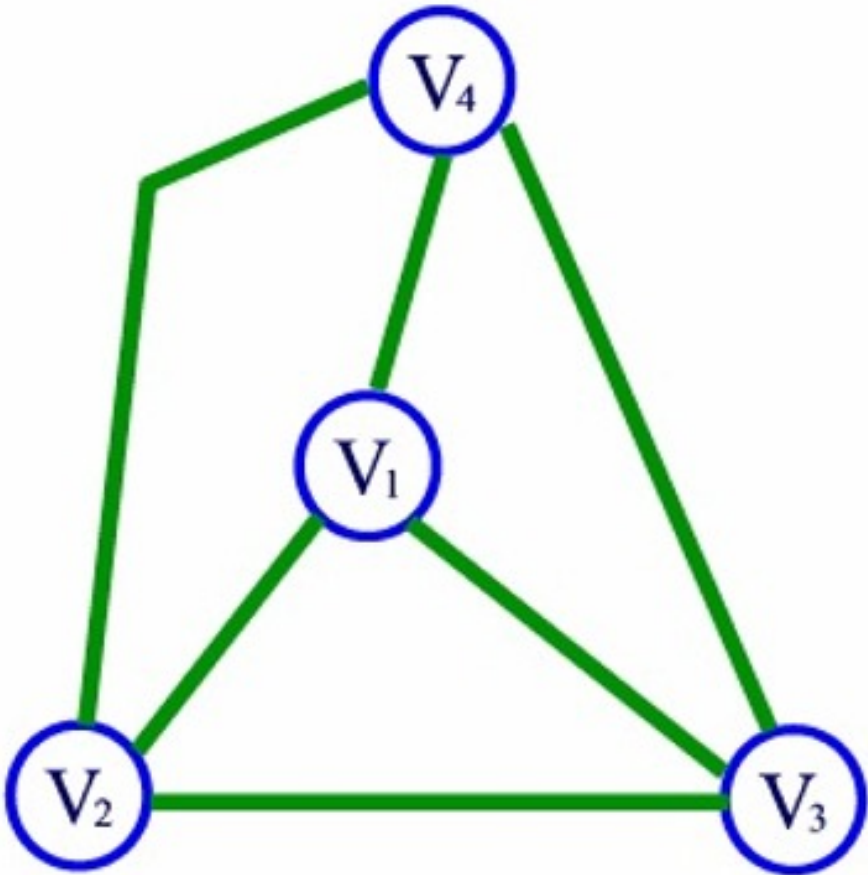
Componentes Principales: Vértices

También conocidos como nodos. Son las entidades fundamentales que almacenan la información.



Componentes Principales: Aristas

También conocidas como enlaces (edges). Son las conexiones tangibles que unen a los vértices.



La Fórmula Fundamental

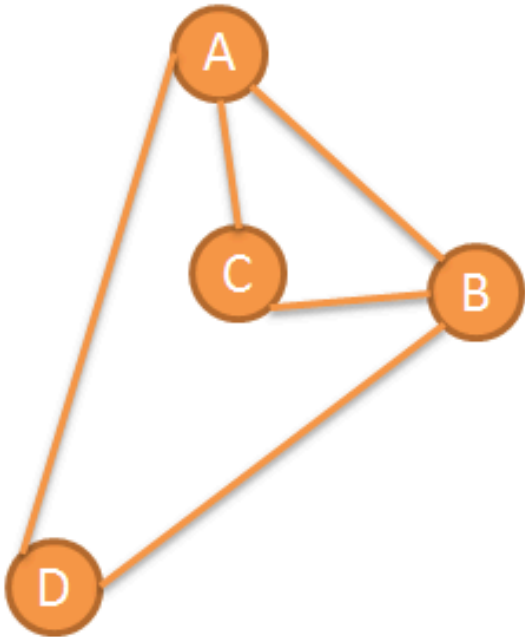
$$G = (V, E)$$

Un Grafo (G) es un conjunto de Vértices (V) y un conjunto de Aristas (Edges).

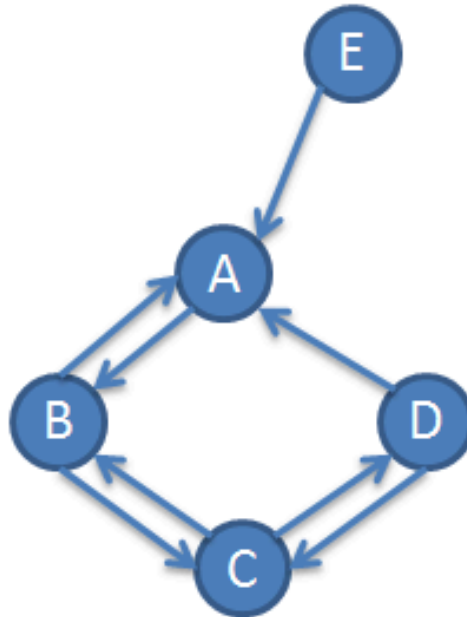
Tipos de Grafos: No Dirigido

Las relaciones son simétricas o bidireccionales por defecto. Una conexión entre A y B se puede recorrer en ambos sentidos.

Grafo No Dirigido



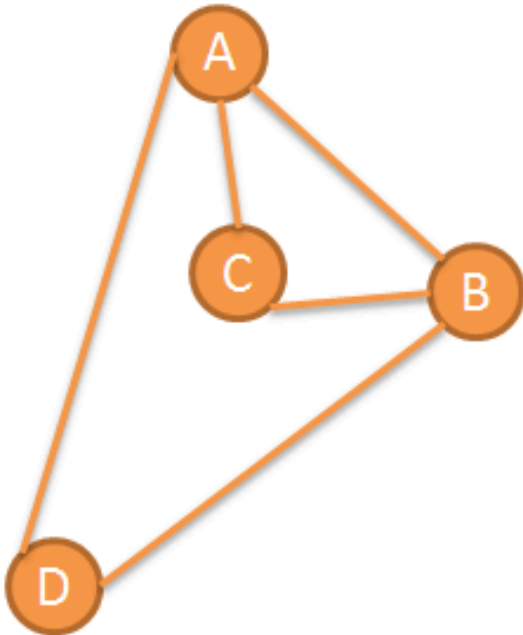
Grafo Dirigido



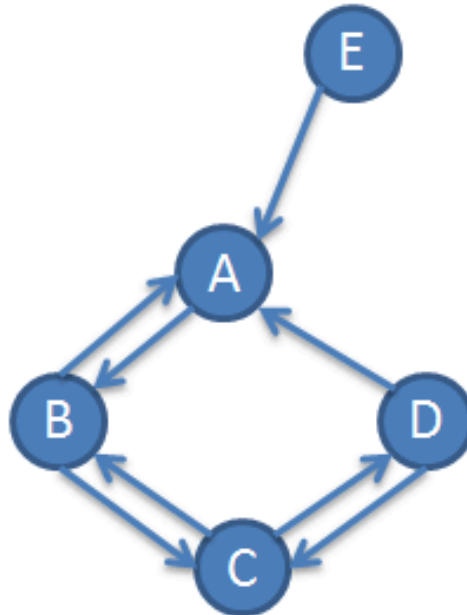
Tipos de Grafos: Dirigido

Las conexiones son unidireccionales. Si hay una arista de D hacia A, no implica que exista de A hacia D.

Grafo No Dirigido



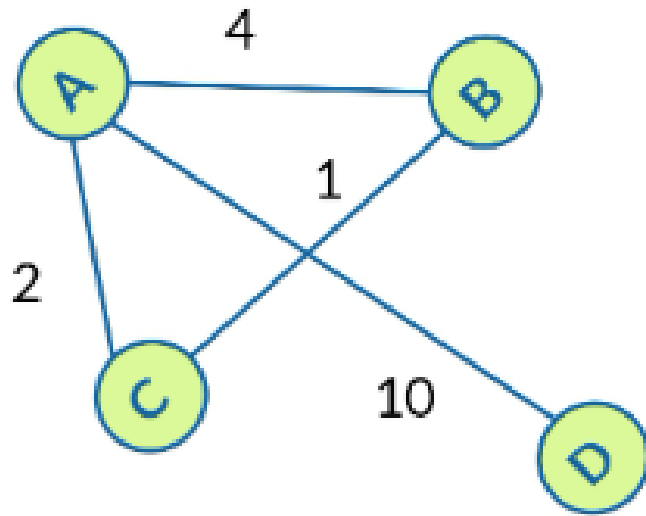
Grafo Dirigido



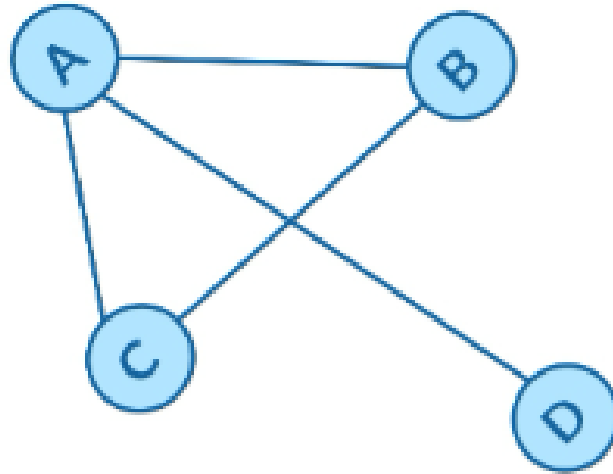
Tipos de Grafos: Ponderado

Cada arista contiene un *peso*. Este peso puede representar costo, distancia, tiempo o capacidad.

Ponderados



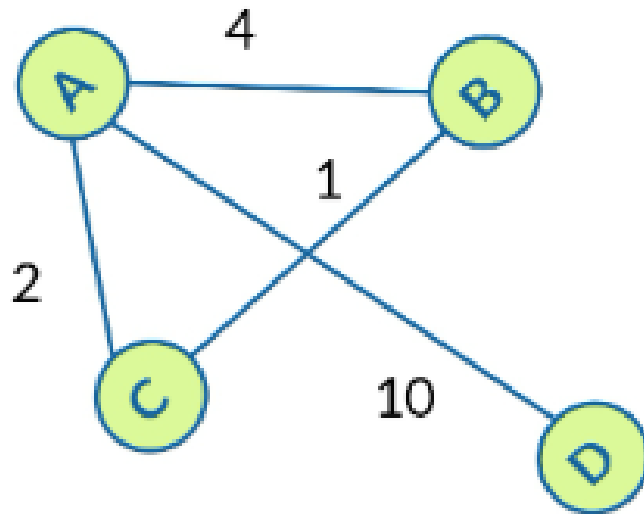
No Ponderados



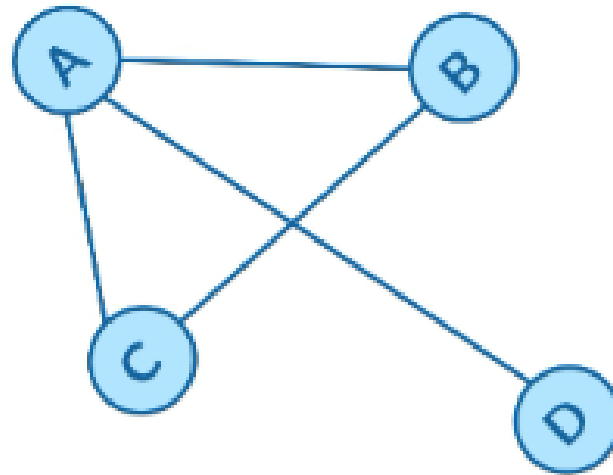
Tipos de Grafos: No Ponderado

Todas las conexiones tienen un costo unitario idéntico. Solo importa si existe o no la conexión.

Ponderados

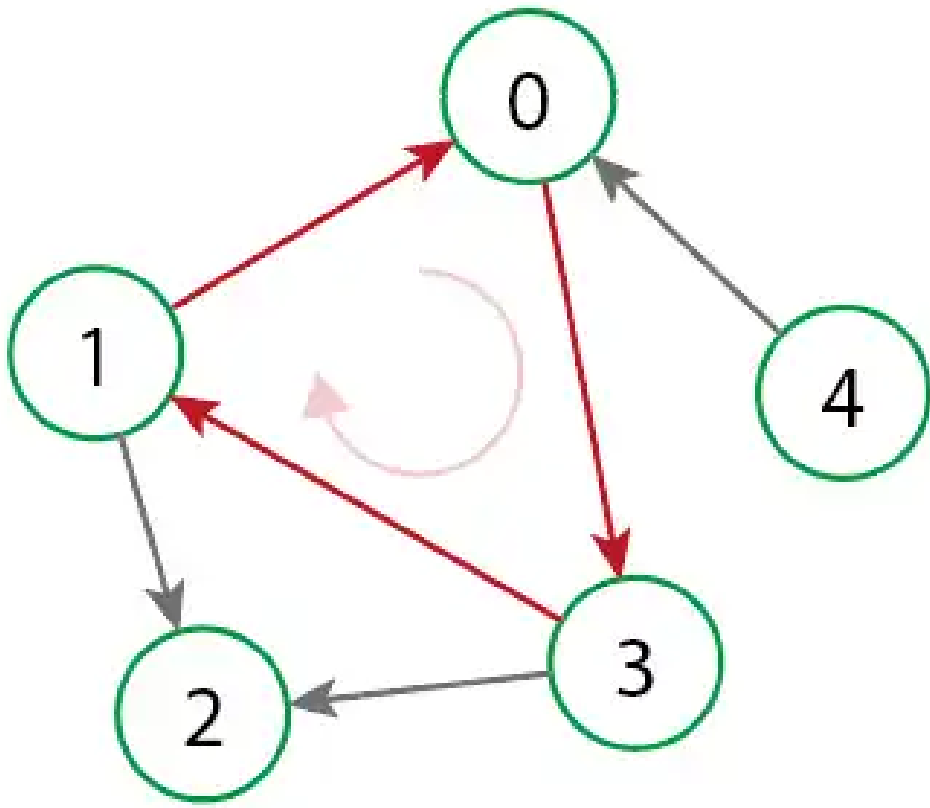


No Ponderados



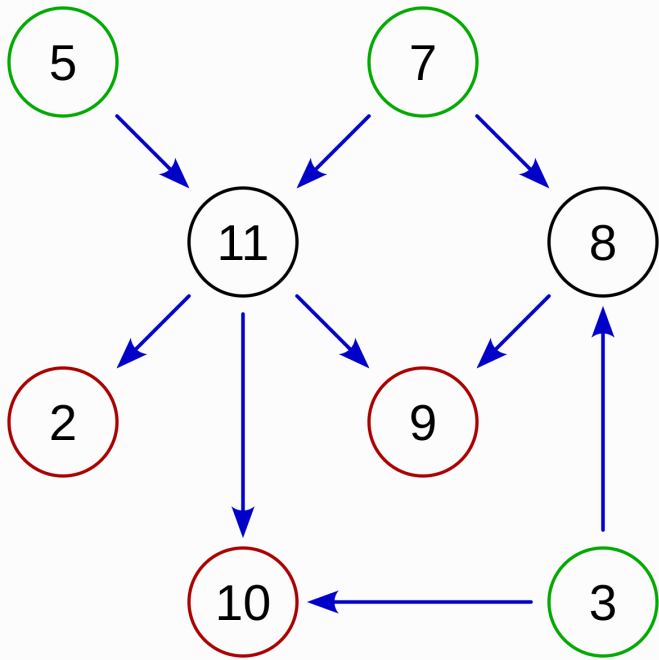
Topología: Ciclos

Un ciclo es una ruta que comienza y termina en el mismo vértice, permitiendo recorrer un bucle infinito.



Topología: Acíclicos

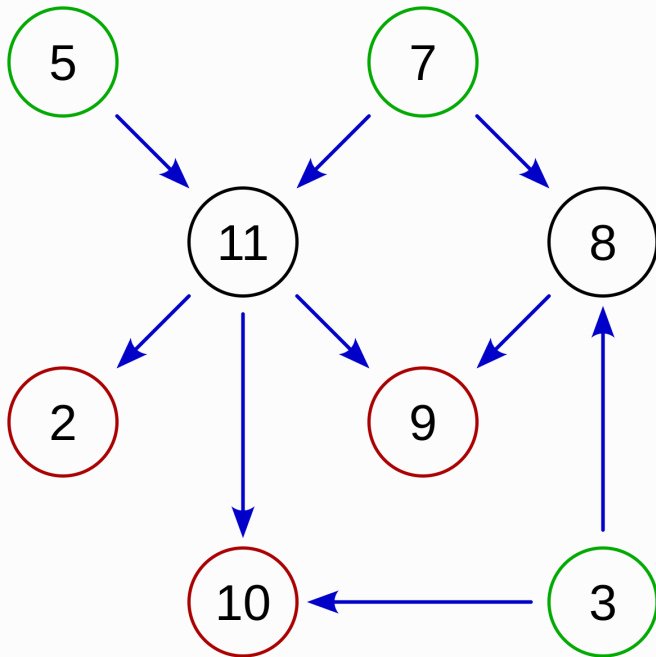
Redes estrictamente sin bucles ni retornos posibles. Una vez que se avanza, no se puede regresar al origen.



DAG

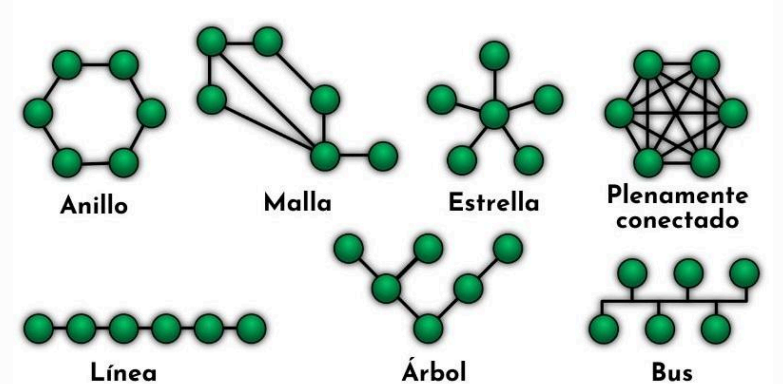
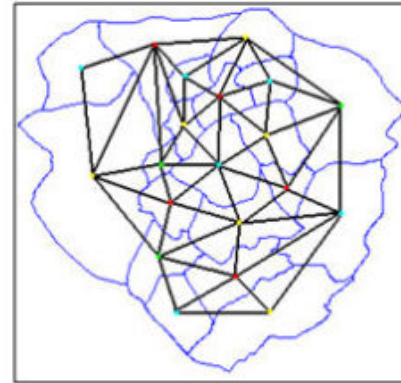
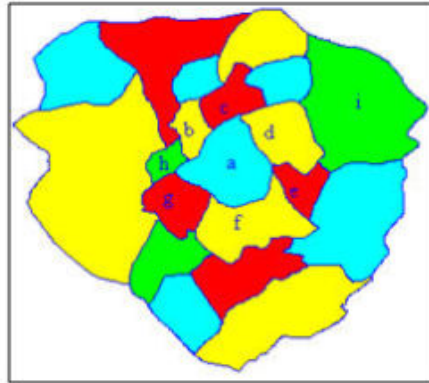
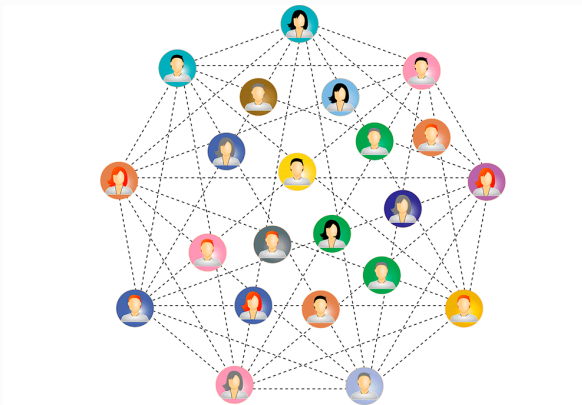
Directed Acyclic Graph (Grafo Acíclico Dirigido).

Es la base algorítmica para la resolución de dependencias, como en los gestores de paquetes o sistemas de compilación.



Aplicaciones Reales

- Redes sociales (amigos, seguidores).
- Sistemas GPS y mapas (rutas más cortas).
- Motores de recomendación.
- Topología de redes de computadoras.



2. Propiedades y Memoria

El Desafío de la Memoria

¿Cómo almacenamos topologías irregulares e interconectadas en una memoria de computadora que es lineal?

Matriz de Adyacencia

Una cuadrícula 2D (matriz cuadrada) que marca con un 1 la existencia de conexiones directas y con un 0 la ausencia de ellas.

Matriz de Adyacencia: Visualización

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

Complejidad Espacial de la Matriz

- $O(V^2)$

Complejidad cuadrática. Es muy costosa en redes grandes porque reserva memoria incluso para conexiones que no existen.

Lista de Adyacencia

Un mapa, diccionario o arreglo donde cada vértice almacena únicamente una lista de sus vecinos directos.

Lista de Adyacencia: Visualización

- **A** $\rightarrow$ [B]
- **B** $\rightarrow$ [A, C]
- **C** $\rightarrow$ [B]

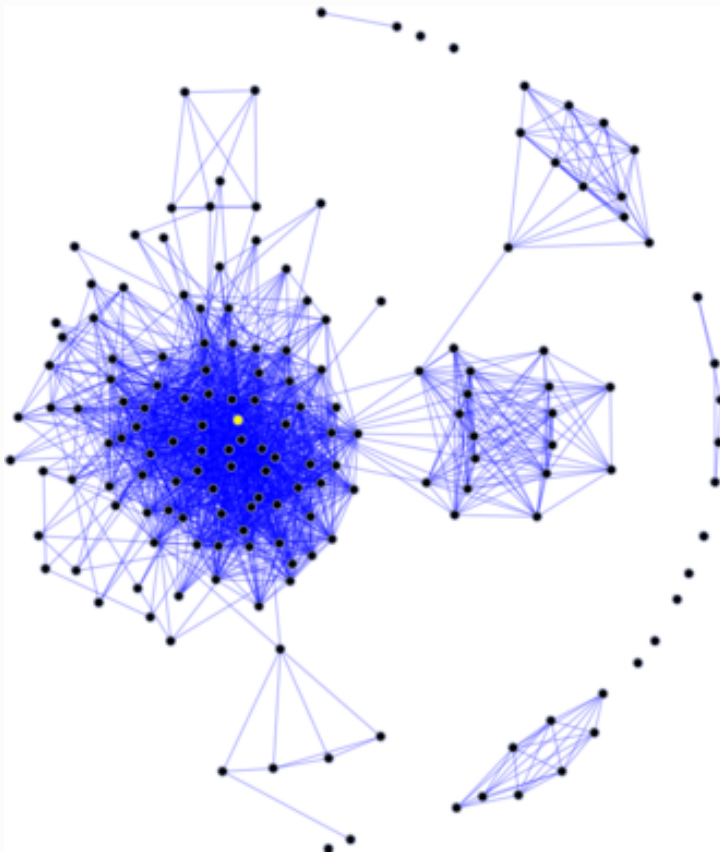
Complejidad Espacial de la Lista

- $O(V + E)$

Complejidad lineal. Altamente eficiente ya que solo ocupa memoria proporcional a las entidades y conexiones reales.

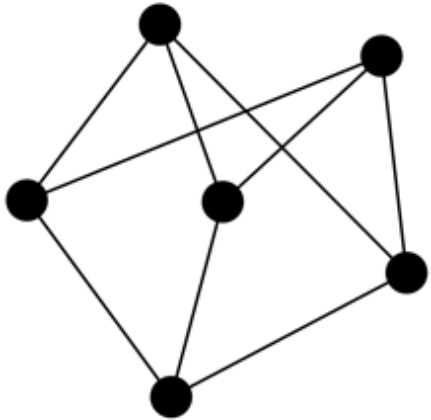
¿Cuándo usar Matriz?

Grafos Densos: Cuando casi todas las conexiones posibles entre vértices existen. Las operaciones de consulta son $O(1)$.



¿Cuándo usar Lista?

Grafos Dispersos: Cuando hay pocos enlaces por nodo (la mayoría de los casos reales). Ahorra cantidades masivas de memoria.



3. Grafos en Código

Implementación: Estructura Base

```
class Grafo:  
    def __init__(self):  
        # Utilizamos un diccionario para la Lista de Adyacencia  
        self.adyacencia = {}
```

Implementación: Insertando Entidades

```
def agregar_vertice(self, v):  
    if v not in self.adyacencia:  
        self.adyacencia[v] = []
```

Implementación: Creando Enlaces

```
def agregar_arista(self, origen, destino):  
    # Grafo dirigido  
    self.adyacencia[origen].append(destino)  
  
    # Si fuera NO dirigido, agregamos también:  
    self.adyacencia[destino].append(origen)
```

Recorridos de un Grafo

Algoritmos para visitar y explorar cada nodo del grafo sistemáticamente sin entrar en ciclos infinitos.

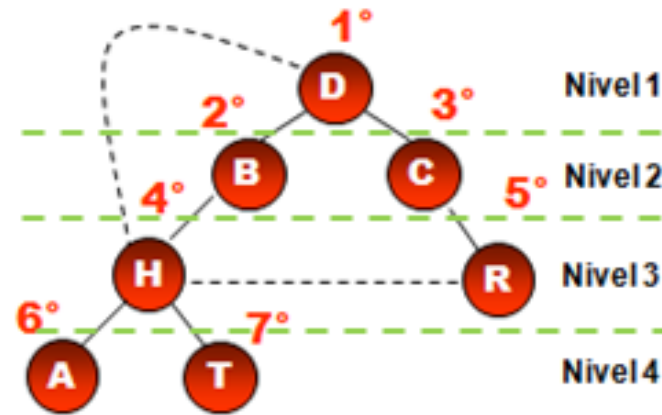
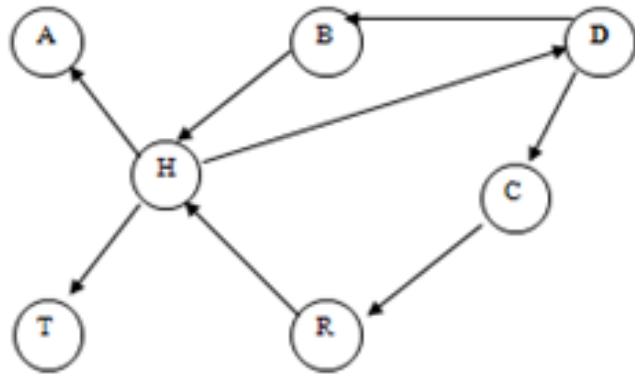
BFS (Búsqueda en Anchura)

Exploración expansiva nivel por nivel. Ideal para encontrar la ruta más corta en grafos no ponderados.

Criterios

1. Pesos uniformes en las aristas (para pesos no uniformes se utiliza el algoritmo de *Dijkstra*).
2. Control obligatorio de nodos visitados.
3. Cobertura de componentes desconectadas.
4. Estructura de datos y consumo de memoria (Cola FIFO).
5. Dirección de las aristas.

BFS



BFS: Implementación (Usando Cola)

```
from collections import deque

def bfs(grafo, inicio):
    visitados = set()
    cola = deque([inicio])

    while cola:
        nodo = cola.popleft()
        if nodo not in visitados:
            visitados.add(nodo)
            cola.extend(grafo[nodo])
```

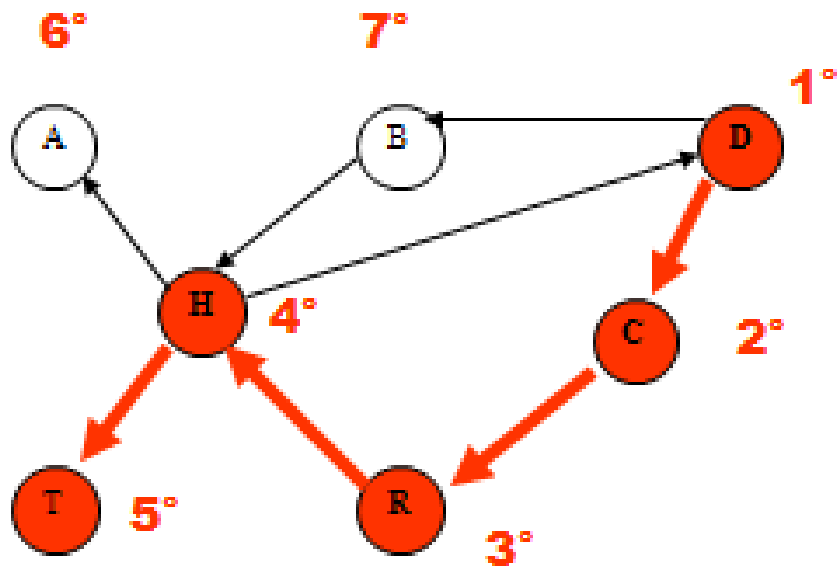
DFS (Búsqueda en Profundidad)

Exploración ramificada hasta el fondo. Avanza lo más lejos posible por una rama antes de retroceder (*backtracking*).

Criterios

1. Estructura de Datos (LIFO).
2. Sin garantía de ruta más corta.
3. Control estricto de nodos y estados (detección de ciclos).
4. Ideal para orden topológico (DAG) y componentes conexas (algoritmos *Tarjan* o *Kosaraju*).
5. Consumo de memoria según la profundidad.
6. Cobertura de componentes aisladas.

DFS



DFS: Implementación (Recursividad/Pila)

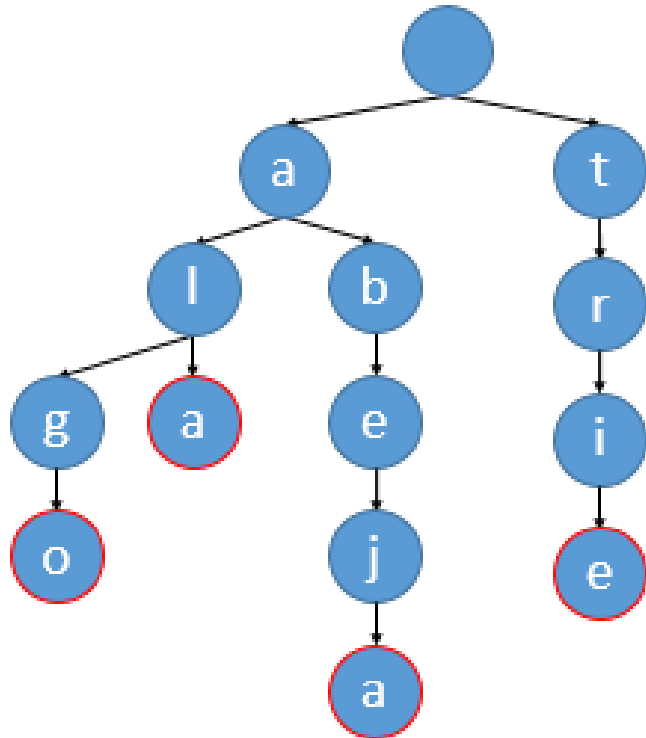
```
def dfs(grafo, inicio, visitados=None):  
    if visitados is None:  
        visitados = set()  
  
    visitados.add(inicio)  
  
    for vecino in grafo[inicio]:  
        if vecino not in visitados:  
            dfs(grafo, vecino, visitados)  
  
    return visitados
```

4. El Árbol Trie

¿Qué es un Trie?

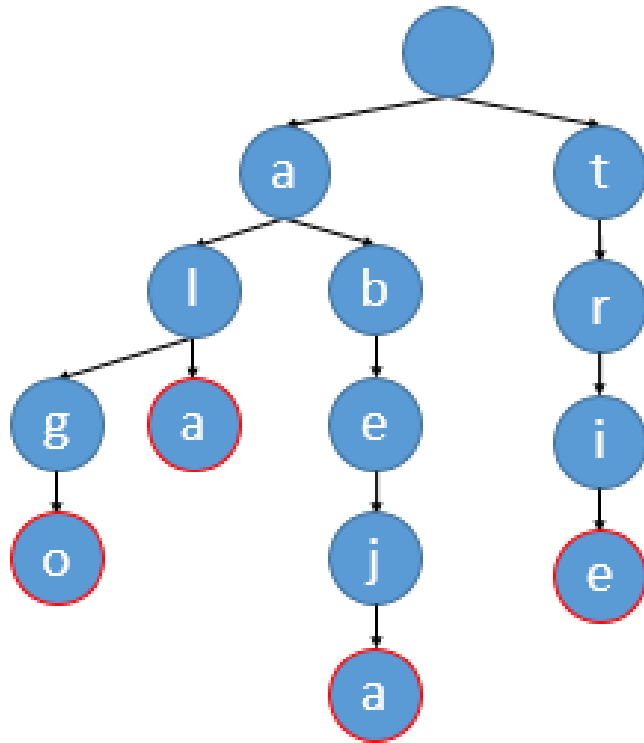
También llamado *Prefix Tree* (Árbol de Prefijos).

Es un tipo especial de árbol n-ario optimizado específicamente para almacenar y buscar cadenas de texto (*strings*).



Propiedad Fundamental

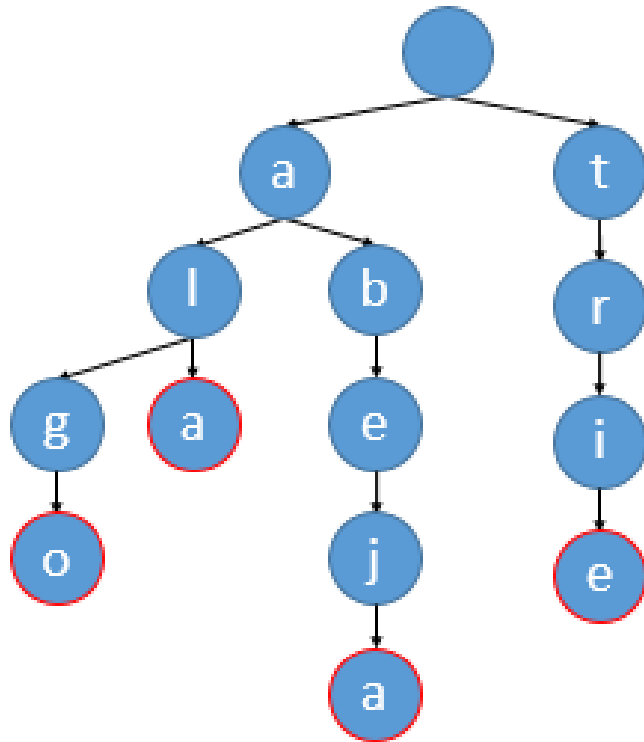
A diferencia de los árboles de búsqueda binaria, los nodos no almacenan la clave entera. Cada nodo representa **un único carácter o letra**.



Rutas Compartidas

Las palabras que comparten un mismo prefijo comparten también los mismos nodos iniciales en la estructura de memoria.

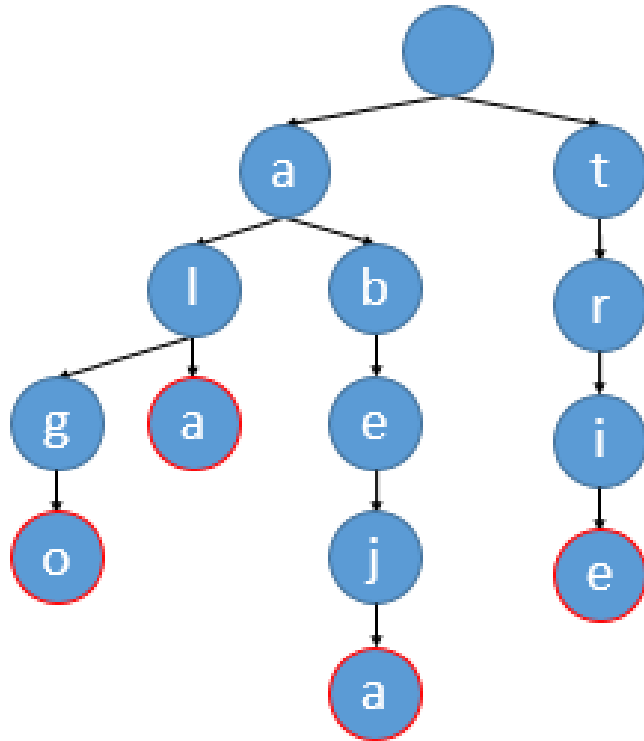
*Ejemplo: **g**ato y **g**anar comparten la **g** y la **a**.*



Complejidad Temporal del Trie

- $O(L)$

La búsqueda es extremadamente veloz y solo depende de la longitud L de la palabra que estamos buscando, independientemente de cuántos millones de palabras existan en el árbol.



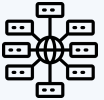
Casos de Uso Reales



Autocompletado predictivo en buscadores.



Correctores ortográficos.



Enrutamiento de direcciones IP en redes.



Juegos como Scrabble o Boggle.

5. Tries en Código

El Nodo Trie

Cada nodo debe tener referencias a sus hijos y una bandera booleana para indicar si ahí termina una palabra válida.

```
class NodoTrie:
    def __init__(self):
        self.hijos = {} # Diccionario letra -> NodoTrie
        self.fin_palabra = False
```

La Clase Trie

```
class Trie:
    def __init__(self):
        # La raíz siempre es un nodo vacío
        self.raiz = NodoTrie()
```

Inserción de Palabras

```
def insertar(self, palabra):  
    nodo = self.raiz  
    for char in palabra:  
        if char not in nodo.hijos:  
            nodo.hijos[char] = NodoTrie()  
        nodo = nodo.hijos[char]  
  
    # Marcamos el último nodo como fin de palabra  
    nodo.fin_palabra = True
```

Búsqueda de Palabras Exactas

```
def buscar(self, palabra):  
    nodo = self.raiz  
    for char in palabra:  
        if char not in nodo.hijos:  
            return False  
        nodo = nodo.hijos[char]  
  
    return nodo.fin_palabra
```

Búsqueda de Prefijos (Autocompletado)

```
def empieza_con(self, prefijo):  
    nodo = self.raiz  
    for char in prefijo:  
        if char not in nodo.hijos:  
            return False  
        nodo = nodo.hijos[char]  
  
    return True # Si llegamos aquí, el prefijo existe
```

Resumen Final

- **Grafos:** Flexibilidad total para modelar relaciones. Optimizados con listas de adyacencia.
- **Tries:** Estructura rígida pero inigualable en velocidad y memoria para manejar prefijos y diccionarios de texto.